

Infraestructura de Agentes — Análisis completo de la Sesión 14

Fuente base: *Agentes IA — Agents Construction* — Módulo 5 (Herramientas para Orquestación), Programa en Diseño e Implementación de Agentes IA, UTEC Posgrado. Dictada por MEng. Boris Alzamora (mismo docente de las Sesiones 9 y 13).

Nota técnica: el PDF original (`SES14_M5_Langchain_Agents_Infra.pdf` , exportado desde PowerPoint) tiene muy poca capa de texto — la mayoría del contenido son diagramas de arquitectura. Este documento se generó extrayendo el texto disponible y **renderizando e interpretando visualmente** cada diapositiva (17 en total).

Complementado con: investigación propia sobre el protocolo A2A (Agent2Agent, Google, 2025) y su relación con MCP (Model Context Protocol, Anthropic, 2024), y sobre el patrón *Agent as Tool*.

1. Objetivos y Agenda

Objetivos declarados:

1. Comprender **componentes NO-AI estratégicos** — es decir, todo lo que rodea al LLM (*Large Language Model*, modelo de lenguaje de gran escala) en un agente de producción y que **no es** el modelo en sí.
2. Comprender la **Arquitectura de Infraestructura Enterprise** de un agente.

Agenda — Parte 1:

#	Tema
1	Revisión de un agente: *knowledge, tools, memory*
2	Opciones de Front End
3	Telemetría: LangSmith

Agenda — Parte 2:

#	Tema
4	Arquitectura *Agent as Tool*
5	*Multi agent Architecture*

Idea central de la sesión: un LLM que llama herramientas es solo el núcleo de un agente. Llevarlo a producción en una empresa exige una capa completa de infraestructura que **no**

tiene inteligencia artificial — autenticación, balanceo de carga, caché, filtros de datos sensibles, *guardrails*, observabilidad, seguridad y evaluación continua. Esta sesión es, en esencia, el paso de "agente que funciona en mi laptop" a "agente que una empresa puede operar con confianza".

2. El agente y sus 8 componentes "NO-AI" de soporte

El material presenta un diagrama radial con el **Agent** en el centro (rodeado de **Knowledge**, **Memory** y **Tools** — los tres bloques "AI-native" ya vistos en sesiones previas) y, alrededor, **8 componentes de infraestructura que no son inteligencia artificial** pero son indispensables para operar el agente en un entorno empresarial real:

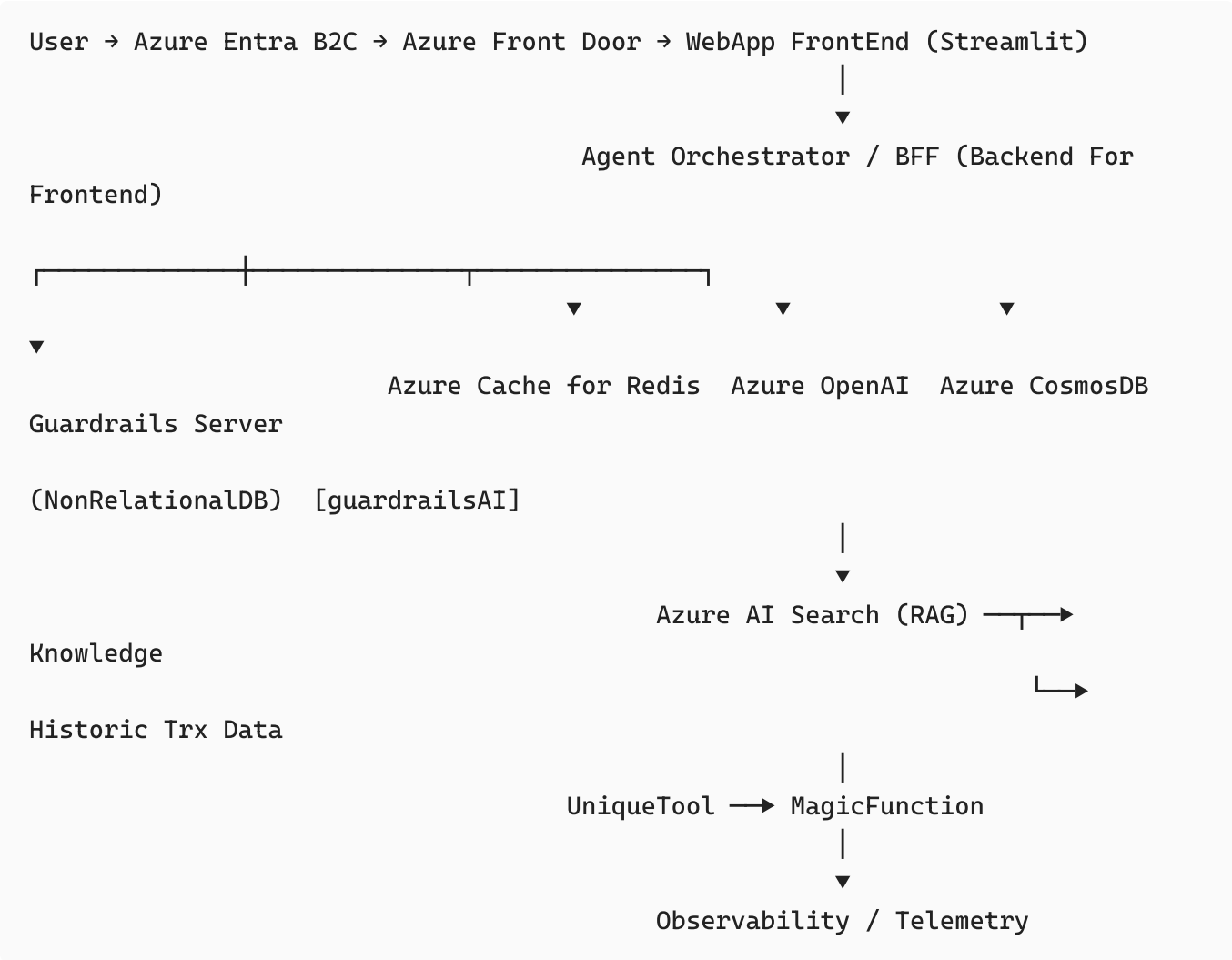
Componente	Qué resuelve
Filtros PII (<i>Personally Identifiable Information</i> , información de identificación personal)	Detecta y redacta/enmascara datos personales sensibles (nombres, DNI, tarjetas) antes de que entren o salgan del modelo.
Load Balancing (balanceo de carga)	Distribuye las peticiones entre múltiples instancias del modelo/servicio para soportar volumen y evitar puntos únicos de falla.
Guardrails (barreras de seguridad)	Valida las entradas y salidas del agente contra reglas de negocio, contenido prohibido o formatos esperados — independiente del criterio del LLM.
CiberSeguridad	Protege el agente como cualquier sistema expuesto: autenticación, autorización, cifrado, protección perimetral.
Observabilidad / Telemetría	Registra qué hizo el agente, con qué latencia, con qué costo y con qué resultado — necesario para depurar y auditar decisiones que el LLM tomó de forma autónoma.
Evals & RLHF (<i>Evaluations & Reinforcement Learning from Human Feedback</i> , evaluaciones y aprendizaje por refuerzo con retroalimentación humana)	Mide la calidad de las respuestas del agente de forma sistemática y usa esa retroalimentación para mejorar el sistema con el tiempo.

Por qué esto importa: ninguno de estos 8 componentes hace que el agente sea "más inteligente" — un agente sin ellos puede razonar y llamar herramientas exactamente igual. Lo que aportan es **confiabilidad operativa**: que el agente no filtre datos personales, no se caiga bajo carga, no responda algo prohibido, sea auditable, y mejore con el tiempo. Es la

misma idea que el *Checklist* de Anthropic de la Sesión 15 (§10 de ese análisis) formaliza con su criterio de **"costo del error"**: cuanto más alto ese costo, más de esta infraestructura hace falta antes de confiar en el agente.

3. Arquitectura de referencia Enterprise (Azure)

El material presenta una arquitectura de referencia completa sobre **Microsoft Azure**, mostrada dos veces con variantes (páginas 6 y 11), y una tercera vez ya evolucionada con el patrón *Agent as Tool* (página 13, ver §5). Reconstruida por capas:



Capa	Componentes	Rol
Identidad y borde	Azure Entra B2C (identidad de clientes), Azure Front Door	Autenticación de usuarios finales; Front Door aporta <i>content delivery</i> , balanceo de carga, protección DDoS (<i>Distributed Denial of Service</i> , denegación de servicio distribuida),

Capa	Componentes	Rol
		WAF (<i>Web Application Firewall</i> , cortafuegos de aplicaciones web) y forzado de HTTPS.
Front End	Streamlit , WebApp FrontEnd	Interfaz de usuario del agente — mismo framework usado en el ejercicio <i>KratosAgent</i> (§4).
Orquestación	Agent Orchestrator — BFF (<i>Backend For Frontend</i> , patrón de backend dedicado a un frontend específico)	El punto central que recibe la petición del frontend y coordina el resto de servicios — el "cerebro operativo" del agente en producción.
Estado y memoria	Azure Cache for Redis , Azure CosmosDB (NoSQL)	Caché de baja latencia y base de datos no relacional para persistir conversaciones/estado del agente.
Modelo	Azure OpenAI	El LLM servido dentro del perímetro de Azure (control de datos, cumplimiento normativo).
Conocimiento (RAG)	Azure AI Search (RAG) → Knowledge , Historic Trx Data	Recuperación semántica sobre documentos de la empresa y datos históricos de transacciones — el mismo patrón RAG de la Sesión 13, aplicado a datos corporativos.
Herramientas	Guardrails Server [guardrailsAI] , UniqueTool , MagicFunction	Validación de entradas/salidas (biblioteca open-source Guardrails AI) y herramientas de negocio específicas del caso de uso.
Observabilidad	Observability / Telemetry (evolucionado a LangSmith en la versión de la página 11)	Cierre del ciclo: todo lo que pasó por el orquestador queda registrado para trazabilidad y depuración.

Diferencia entre la versión de la página 6 y la de la página 11: la página 11 reemplaza el bloque genérico *Observability Telemetry* por **Log Analytics Workspace + LangSmith (observability)** — es decir, muestra la telemetría nativa de Azure combinada con la telemetría especializada de LangChain, el mismo LangSmith que se usó para evaluar RAG en la Sesión 13.

4. Del *quickstart* a producción — el ejemplo *KratosAgent*

El material contrasta dos niveles de madurez con dos diagramas consecutivos:

4.1 Nivel *quickstart* (tutorial básico)

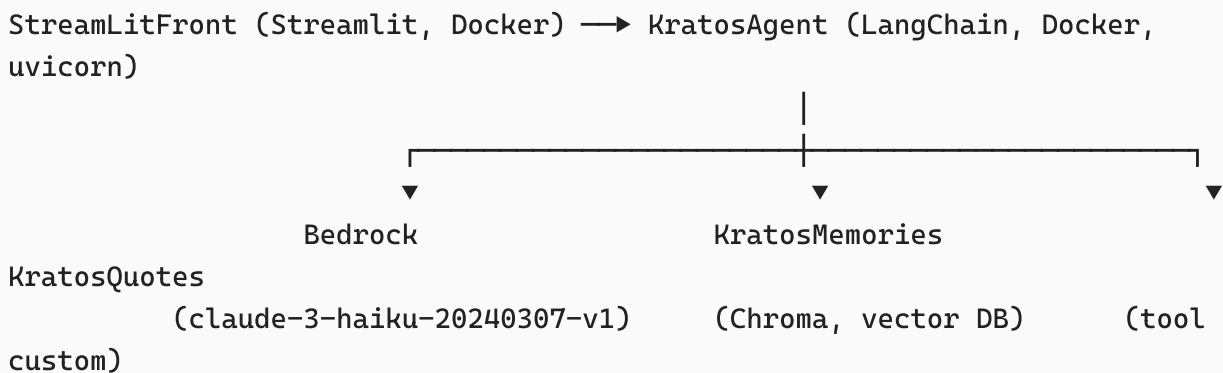
Un diagrama dibujado a mano muestra el patrón mínimo LangChain + Streamlit:

```
Backend: LangChain (LLM framework) —Input/Output→ OpenAI → Large Language Model
Frontend: Streamlit (Web framework) —Input/Output→ Quickstart App → User
```

Con una nota explícita: **"Modules not used in this tutorial"** — Memory, Prompt templates, Indexes, Chains, Agents, Callbacks. Es decir, el ejemplo más básico de LangChain+Streamlit **no usa ni memoria, ni cadenas, ni agentes** — es una sola llamada al modelo por turno. Sirve como punto de partida antes de agregar cualquiera de esos módulos.

4.2 Nivel producción — *KratosAgent*

Un segundo diagrama muestra un caso real ya containerizado:



Lectura de este diagrama: cada componente corre en su propio contenedor **Docker** (aislamiento y despliegue reproducible), el agente se sirve con **uvicorn** (servidor ASGI para Python), el LLM es **Claude 3 Haiku** servido vía **Amazon Bedrock** (no la API directa de Anthropic — otra forma de mantener el modelo dentro del perímetro cloud de la empresa, igual que Azure OpenAI en §3), la memoria semántica usa **Chroma** (la misma base vectorial de la Sesión 13), y hay un ícono de **Jenkins** (herramienta de CI/CD) señalando que el despliegue está automatizado. Este es, en esencia, el mismo proyecto de referencia (`agents26_m5s14-main`, con `kratos_agent.py`, `feedquotes.py`, `kratos.prompt`) que acompaña el material del curso.

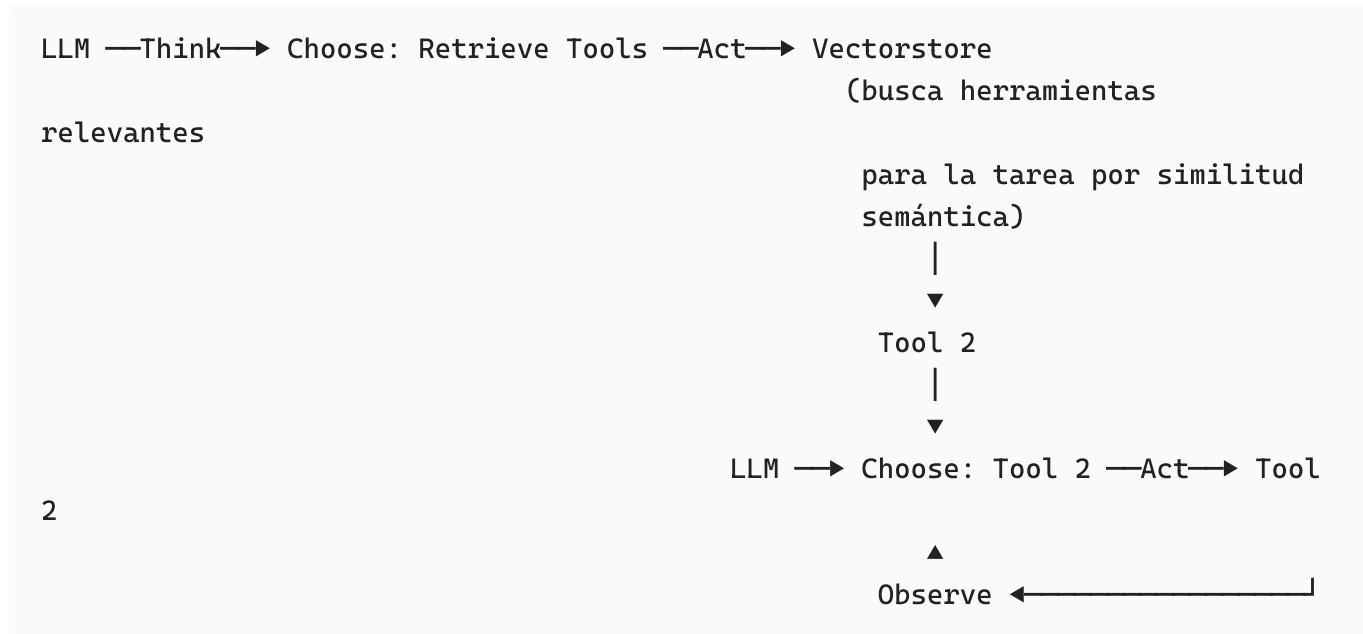
5. Arquitectura *Agent as Tool*

"Asuma que un agente una vez instanciado podría ser invocado como una herramienta como parte de un método, y este mismo ser declarado como herramienta de otro agente."

Así, logramos que un agente se defina como una herramienta para otro agente, en esa forma iniciamos una colaboración entre ellos."

Este es el patrón que Anthropic llama **Orchestrator-Workers** en su framework *Building Effective Agents* (ver JUSTIFICACION_AGENTE_VS_WORKFLOW.md en la raíz de este módulo): un agente orquestador no necesita saber "cómo" resuelve cada sub-tarea — solo necesita poder invocar a otro agente (ya instanciado, con su propio razonamiento interno) exactamente igual que invocaría cualquier otra tool.

El diagrama de recuperación dinámica de herramientas (página 12), reconstruido:



Por qué esto no es solo un detalle técnico: conecta directamente con la advertencia de la Sesión 15 (§6 de ese análisis, cita de Chip Huyen): *"a más herramientas se otorgan al agente, más difícil será usarlas eficientemente"*, porque sus descripciones podrían no encajar en el contexto del modelo. La solución que muestra este diagrama es **no darle todas las tools al LLM de una vez**: en su lugar, las tools mismas están indexadas en un *vectorstore*, y el LLM primero **busca semánticamente cuáles son relevantes** para la tarea actual (Retrieve Tools) antes de decidir cuál invocar — un patrón de **recuperación dinámica de herramientas** que escala a decenas o cientos de tools sin saturar el contexto.

Aplicación a la arquitectura Enterprise (página 13): el material muestra la misma arquitectura Azure de §3 evolucionada — se agrega un nodo AgentAsTool que a su vez invoca a **otro** Agent Orchestrator (BFF), y Azure OpenAI gana (ContentFilters and PromptShields) explícitos. Es decir: un orquestador puede invocar a otro orquestador completo como si fuera una tool más — composición recursiva de agentes.

6. Protocolo A2A (Agent2Agent)

El material introduce el protocolo con una diapositiva de referencia: dos agentes (Agent 1 , Agent 2) conectados entre sí por **A2A**, cada uno con sus propias herramientas conectadas vía **MCP** (*Model Context Protocol*, protocolo estándar de Anthropic — noviembre 2024 — para que un modelo/agente consuma herramientas y fuentes de datos externas de forma uniforme: bases de datos, APIs web, sistema de archivos, GitHub, Slack, etc.).

Investigación complementaria — MCP vs. A2A, quién resuelve qué: son protocolos **complementarios**, no competidores, y operan en capas distintas del stack de un agente:

- **MCP** (Anthropic, nov. 2024) es un protocolo **vertical**: conecta **un** agente con sus herramientas y fuentes de datos (bases de datos, APIs, archivos). Es la capa "agente ↔ herramienta".
- **A2A** (*Agent2Agent*, Google, abril 2025) es un protocolo **horizontal**: permite que **un agente completo** se comuniquen y delegue tareas a **otro agente** — potencialmente construido con un framework distinto, por otro equipo o incluso otra empresa — sin que ninguno necesite conocer los detalles internos (herramientas, memoria, modelo) del otro. Los agentes se descubren mutuamente mediante **Agent Cards** (un documento que describe las capacidades de un agente) y se comunican vía tareas y artefactos sobre HTTP/S + JSON-LD.

En un sistema multiagente de producción, ambos coexisten: cada agente usa **MCP** para hablar con sus propias herramientas, y **A2A** para coordinarse con otros agentes — la diferencia conceptual con el patrón *Agent as Tool* (§5) es que en *Agent as Tool* un agente **conoce e invoca directamente** a otro (acoplamiento fuerte, mismo proceso/framework), mientras que A2A está diseñado para agentes **desacoplados e interoperables entre organizaciones**.

7. Laboratorio y tarea de la sesión

Actividad	Instrucción
Lab — "Aterrizando a proyectos"	Asumir un equipo editorial de una revista de tecnología que aborda el impacto de la IA en la fuerza laboral. Implementar un agente responsable de la edición del artículo, apoyado de otro agente investigador y otro editor , ambos invocados como <i>tools</i> (patrón <i>Agent as Tool</i> , §5).
Tarea grupal — "Agent Solution"	Sustentar el planteamiento de arquitectura de solución del proyecto grupal por Workflow, Agent o MultiAgent (la misma clasificación de JUSTIFICACION_AGENTE_VS_WORKFLOW.md), y entregar la arquitectura actualizada pensando en la infraestructura alrededor de la

Actividad	Instrucción
	implementación agéntica (los componentes de §2-3). Fecha límite: 07/08.

8. Síntesis — lo que hay que llevarse de esta sesión

1. **Un agente de producción es mucho más que el LLM que razona y llama tools.** Los 8 componentes "NO-AI" (PII, load balancing, guardrails, ciberseguridad, observabilidad, evals/RLHF...) son los que determinan si una empresa puede confiar en operar ese agente.
2. **La arquitectura Enterprise de referencia** (Azure, con equivalentes directos en AWS — ver también §18 del análisis de la Sesión 15) sigue siempre el mismo patrón: borde con auth+WAF, front end desacoplado, un orquestador (BFF) central, estado/caché, el LLM, una capa de conocimiento (RAG) y una capa de observabilidad que cierra el ciclo.
3. **El patrón *Agent as Tool*** convierte a un agente completo en una herramienta más de otro agente — es la implementación concreta del patrón *Orchestrator-Workers* de Anthropic, y permite composición recursiva de agentes.
4. **Cuando un agente tiene demasiadas herramientas**, la solución no es reducirlas a la fuerza sino **indexarlas en un vectorstore y recuperar dinámicamente solo las relevantes** para cada tarea — evita saturar el contexto del modelo con descripciones de tools que no se van a usar.
5. **MCP y A2A resuelven problemas distintos y se combinan:** MCP conecta un agente con sus propias herramientas; A2A conecta agentes completos entre sí, incluso entre organizaciones distintas.
6. **La infraestructura no es opcional a partir de cierta escala** — es, en la práctica, la respuesta operativa al criterio de "costo del error" del checklist de Anthropic (Sesión 15): cuanto más alto el riesgo de un error del agente, más de esta capa de soporte hace falta antes de confiar en él en producción.

9. Referencias

Del material original:

- Diagramas propios del curso — arquitectura Enterprise en Azure, diagrama radial de componentes NO-AI, ejemplo *KratosAgent*, patrón *Agent as Tool*, protocolo A2A.
- Guardrails AI — librería de validación de entradas/salidas de LLM. guardrailsai.com

- LangSmith — plataforma de observabilidad de LangChain (ya introducida en la Sesión 13 para evaluación de RAG).

Investigación complementaria (añadida en este documento):

- Anthropic — *Model Context Protocol (MCP)*, noviembre 2024. Protocolo estándar para conectar agentes con herramientas y fuentes de datos externas. modelcontextprotocol.io
- Google — *Agent2Agent Protocol (A2A)*, abril 2025. Protocolo abierto de interoperabilidad agente-a-agente entre frameworks y organizaciones distintas. a2a-protocol.org
- Arco interno del curso: Sesión 13 (RAG con Chroma) — mismo patrón de recuperación semántica aplicado aquí tanto al Azure AI Search RAG de la arquitectura Enterprise como al *retrieval* dinámico de herramientas del patrón *Agent as Tool*. Sesión 15 (LangGraph MultiAgent) — el *checklist* de Anthropic citado en §2 y §8 se desarrolla en detalle en `Sesion15_LangGraph_MultiAgent_ANALISIS_COMPLETO.md`.

Documento generado a partir del PDF de la Sesión 14 (Módulo 5, UTEC Posgrado) — texto extraído + diapositivas gráficas interpretadas visualmente — más investigación propia sobre MCP, A2A y el patrón Agent as Tool.